Evidence-Grounded Clinical Trial Eligibility Screening with a Fine-Tuned Small Language Model: Low-Rank Adaptation, Key–Value Caching, Tiled Attention, and Speculative Decoding

Maryam Fatima
Department of Computer Science
FAST National University of Computer and Emerging Sciences, Islamabad
maryam.faatim4@gmail.com

***Abstract*—Matching a patient to a clinical trial requires judging each inclusion and exclusion criterion against a free-text patient description and citing the sentences that support the judgment. This paper presents an end-to-end, reproducible system built on a small open-weight Transformer, Qwen2.5-1.5B-Instruct, fine-tuned with low-rank adaptation (LoRA) on the TrialGPT Criterion Annotations dataset using a patient-grouped split of 37 training, 8 validation, and 8 test patients. The model returns one of six expert label categories together with the identifiers of the supporting sentences. On the held-out test patients, fine-tuning raises accuracy from 0.117 to 0.743, macro-F1 from 0.142 to 0.536, and evidence F1 from 0.124 to 0.546. The dominant remaining error is treating missing information as proof that an exclusion criterion does not apply. Beyond accuracy, three inference mechanisms are implemented from scratch and validated against references. A key–value (KV) cached greedy decoder is token-identical to an uncached decoder and up to 8.0 times faster. A tiled attention with online softmax agrees with a float64 reference to within one part in a thousand in half precision and uses 125 times less memory at 4096 tokens. A greedy speculative decoder preserves the output exactly but, on a free T4 graphics processor, gives no speedup; the best measured value is 1.00 times. A browser application exposes screening, evidence inspection, report export, and a live benchmark of the decoding modes. The system is an educational prototype and not a clinical decision tool.**

***Index Terms*—clinical trial matching, eligibility criteria, low-rank adaptation, KV cache, online softmax, speculative decoding, evidence retrieval**

I. Introduction

Deciding whether a patient may enter a clinical trial requires reading the patient description and checking it against every inclusion and exclusion criterion of the trial. The task is slow for clinicians and hard for automatic systems for three reasons. First, criteria are free text and often compound. Second, the patient record rarely mentions every fact that a criterion needs, so a system must be able to state that information is missing instead of guessing. Third, a decision is only useful if it points to the sentences that justify it.

Recent work has shown that large language models can perform this criterion-level matching when prompted [1], and that open models adapted with supervised data can approach proprietary ones [2]. Those results, however, rely on large proprietary models or substantial hardware. This paper asks a practical question: how far can a 1.5-billion-parameter open-weight Transformer be taken on a free graphics processing unit (GPU) when fine-tuning, evaluation, inference acceleration, and a user-facing application must all be implemented and verified by one person?

The contributions are as follows.

- A LoRA-adapted Qwen2.5-1.5B-Instruct model that, given a patient note, a criterion type, and a criterion text, returns a structured label and supporting sentence identifiers. It is evaluated before and after fine-tuning on identical examples under a patient-grouped protocol.
- An evaluation that preserves all six expert label categories, defines an explicit policy for records without evidence, and analyzes errors, with emphasis on missing information.
- From-scratch implementations of a KV-cached decoder, tiled online-softmax attention, and greedy speculative decoding. Each is validated for exact or numerically bounded agreement with a reference and measured honestly, including a case where it does not help.
- A browser application with per-criterion evidence, a rule-based overall outcome, report export, and a live comparison of decoding modes.
- A reproducible package: code, split manifest, logs, and an adapter download.

Section II reviews related work. Section III describes the data, model, algorithms, and application. Section IV gives the experimental setup, Section V presents and discusses the results, and Sections VI and VII state the limitations and conclusions.

II. Related Work

Trial matching with language models. TrialGPT [1] decomposes matching into retrieval, criterion-level matching with explanations and evidence sentences, and ranking. Its criterion-level annotations by physicians form the dataset used here [3]. Its strongest results use a proprietary model, which

TABLE I
MAIN SOURCES AND THEIR ROLE IN THIS WORK

Source	Contribution	Limit for this task / use here
[1]	Criterion-level matching, evidence, ranking	Proprietary model; dataset source only
[2]	Open models adapted for trial matching	Distilled labels; here expert labels are used
[4], [5]	Low-rank adapters; quantized variant	Quantization unnecessary; LoRA selected
[6], [7], [8]	Online softmax; fused tiled attention	Fused kernels unavailable on T4; algorithm reimplemented
[10], [11]	Draft-and-verify decoding	Gain depends on draft cost; measured here

TABLE II
USABLE ROWS PER LABEL AND SPLIT

Type	Label	Train	Val	Test
Inclusion	included	88	35	27
	not included	13	3	7
	not enough information	109	27	38
	not applicable	5	1	3
Exclusion	excluded	8	2	0
	not excluded	291	101	91
	not enough information	65	26	28
	not applicable	23	11	12
Total rows (patients)		602 (37)	206 (8)	206 (8)

limits local use and reproducibility. Nievas et al. [2] study the distillation of a large model into smaller open models for the same matching problem. The present work follows this direction, but fine-tunes directly on the expert labels and evidence sentences and does not use any model-generated annotation.

Parameter-efficient adaptation. LoRA [4] freezes the pre-trained weights and learns low-rank updates, and QLoRA [5] adds four-bit quantization of the frozen weights to reduce memory further. QLoRA is the safer choice when a model barely fits the GPU. Here the 1.5-billion-parameter model fits in half precision with LoRA, so quantization was not needed and the simpler method was used.

Efficient attention. Softmax over a long row can be computed in a single pass with a running maximum and normalizer [6]. FlashAttention [7] and FlashAttention-2 [8] use this idea inside fused GPU kernels that process tiles in fast on-chip memory. Their speed depends on hardware features that a free Turing-generation GPU lacks, which is why the implementation in this paper is evaluated mainly for correctness and memory.

Efficient decoding. Autoregressive Transformers [9] reuse keys and values of earlier tokens through a cache. Speculative decoding [10], [11] lets a cheap draft model propose several tokens that the target model verifies in one pass. The speedup depends on the draft acceptance rate and on the cost ratio between draft and target.

Table I summarizes how each source informs the design and what limits its direct use.

III. MATERIALS AND METHODS

A. Dataset and Patient-Grouped Split

Only the TrialGPT Criterion Annotations dataset [3] is used. It contains 1,015 annotated patient–criterion pairs from 53 patients, labelled by three physicians [1]. Each row holds a patient note with numbered sentences, a trial identifier and title, the criterion type, the criterion text, the expert eligibility label, and the identifiers of the expert-selected evidence sentences. The dataset also stores predictions and explanations produced earlier by GPT-4 and a flag that marks the dataset's own training subset. These fields were removed immediately after loading, and an assertion verifies that no column whose name begins with "gpt4" remains, so earlier model outputs cannot reach a prompt or an evaluation.

The split follows the common protocol. The unique patient identifiers are sorted, shuffled with the NumPy generator `default_rng(42)`, and assigned in order: the first 37 to training, the next 8 to validation, and the remaining 8 to testing. All rows of a patient belong to exactly one split, set intersections are asserted to be empty, and the manifest with patient identifiers and counts is published. One training row has no criterion text. It was excluded after the split had been computed, logged, and counted, so the prescribed 37/8/8 partition is unchanged. The usable rows number 602, 206, and 206.

All six expert label categories are preserved (Table II). Inclusion criteria use *included*, *not included*, *not enough information*, and *not applicable*; exclusion criteria use *excluded*, *not excluded*, and the same last two. The distribution is strongly imbalanced: the label *not excluded* alone covers 483 of the 1,014 usable rows, whereas *excluded* has 10 and *not applicable* for inclusion has 9. Accuracy alone is therefore a weak indicator, and macro-F1 and per-class scores are reported throughout. The test split contains no *excluded* example.

B. Model Selection and Feasibility

Table III compares the candidates. The target is Qwen2.5-1.5B-Instruct [12], a decoder-only Transformer with 1.544 billion parameters (2.88 GiB in half precision), 28 layers, hidden size 1536, 12 query heads and 2 key–value heads (grouped-query attention [13]), head dimension 128, rotary position embeddings [14], SwiGLU feed-forward layers, RMS normalization, tied input and output embeddings, a vocabulary of 151,936 tokens, a context window of 32,768 tokens, and the Apache 2.0 license. The draft model for speculative decoding is Qwen2.5-0.5B-Instruct (0.494 billion parameters, 0.92 GiB, 24 layers, hidden size 896, 14 query and 2 key–value heads, head dimension 64). Both models use the same tokenizer; this was verified by comparing the full vocabularies and by encoding a sample text with both.

The size of the key–value cache per token is $2\,N_\ell H_{kv} d\,b$ bytes, where N_ℓ is the number of layers, H_{kv} the number of key–value heads, d the head dimension, and b the bytes per element. This gives 28,672 bytes per token for the target and

TABLE III
CANDIDATE OPEN-WEIGHT DECODER MODELS

Model	Params	Licence	Decision
Qwen2.5-1.5B-Instruct	1.54 B	Apache 2.0	Selected as target
Qwen2.5-0.5B-Instruct	0.49 B	Apache 2.0	Selected as draft (same tokenizer)
Llama-3.2-1B-Instruct	≈1.2 B	Community licence, gated	Not used: gated access
Qwen2.5-3B-Instruct	≈3.1 B	Different licence	Not used: weights ≈5.8 GiB, slower training

TABLE IV
FINE-TUNING CONFIGURATION

Setting	Value
LoRA rank r / scale α / dropout	16 / 32 / 0.05
Adapted modules	`q,k,v,o,gate,up,down` projections
Optimizer / weight decay	AdamW / 0
Learning rate / schedule	2×10^{-4} / cosine, 10% warm-up (11 steps)
Epochs / optimizer steps	3 / 114 (38 per epoch)
Effective batch size	16 (micro-batch 1×16 accumulation)
Gradient clipping	1.0
Precision	fp16 autocast, fp32 adapter weights
Maximum length / seed	1,024 tokens / 42

12,288 bytes for the draft in half precision. Grouped-query attention makes the target cache six times smaller than with 12 key–value heads. LoRA training of the target peaked at 7.95 GiB on a GPU with 14.56 GiB of usable memory, so the choice is feasible with margin.

C. Input, Output, and Tokenization

A shared module builds every prompt, so that training, evaluation, and the application cannot drift apart. The system message states that this is an educational screening task, that one criterion must be judged using only the note, that missing facts must never be assumed, and that the answer must be JSON. The user message contains the numbered note sentences, the criterion type and text, the allowed labels for that type, and the required format. The output schema is a JSON object with a label and a sorted list of evidence sentence identifiers:

`{"label": <label>, "evidence": [ids]}`

The Qwen chat template and tokenizer are used. The training target is this JSON string followed by the end-of-sequence token. A parser extracts the first JSON object and accepts it only if the label belongs to the allowed set of the criterion type; otherwise the output is marked invalid, scored as an error, and never repaired.

Prompt plus target lengths in tokens are: training mean 294, 95th percentile 460, maximum 589; validation mean 307, maximum 423; test mean 277, maximum 483. The maximum training length was set to 1,024 tokens, so no example is truncated and no evidence sentence can be removed. The training code raises an error instead of truncating silently, and the application refuses inputs above 1,536 tokens for the same reason. Because the dataset explanations are GPT-4 outputs and are excluded, the model is not trained to write explanations; the application builds them from the label and the cited sentences (Section III-I).

D. Fine-Tuning

LoRA replaces a frozen weight matrix W_0 by

$$W = W_0 + \frac{\alpha}{r} BA, \quad B \in \mathbb{R}^{d_{\text{out}} \times r},\ A \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad (1)$$

where only A and B are trained in (1). The training objective is token-level cross-entropy computed only on the answer tokens; prompt tokens receive the ignore label. Table IV lists the configuration. The adapters add 18,464,768 trainable parameters, 1.18% of the total. Half-precision automatic mixed precision with a gradient scaler was used because the GPU has no bfloat16 support, with the trainable weights kept in single precision. Gradient checkpointing and a micro-batch of one example with 16 accumulation steps were adopted after out-of-memory errors with micro-batches of four and two. One checkpoint is saved per epoch and the epoch is chosen by validation macro-F1, not by loss. At inference the adapter is merged into the base weights. No hyperparameter search was performed.

E. Evaluation Protocol

Classification is evaluated with accuracy, macro-F1, per-class precision, recall, and F1, and confusion matrices, overall and separately for inclusion and exclusion criteria. Macro-F1 averages over the labels that occur in the gold labels or in the predictions, so a label predicted although absent from the gold standard lowers the score; the variant restricted to gold labels is also stored. Invalid outputs are counted as errors and shown in a separate confusion-matrix column.

Evidence is evaluated on sets of sentence identifiers per row. With TP, FP, and FN pooled over rows,

$$P_{\text{micro}} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R_{\text{micro}} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad (2)$$

in (2), and F_1 is the harmonic mean of the two. The policy for empty sets is explicit. A row with no gold and no predicted evidence adds nothing to the pooled counts; a row with no gold evidence but a predicted sentence adds false positives. The macro score over rows with at least one gold sentence gives zero to an empty prediction. A second macro score over all rows gives one to a correct abstention and zero to a spurious citation, and the abstention rate and false-evidence rate on rows without gold evidence are reported directly. The absence of evidence is never treated as proof: *not enough information* is a separate label.

All model comparisons use greedy decoding with repetition penalty 1.0 (the model default of 1.05 would change greedy outputs), at most 64 new tokens, and half precision. The base

Algorithm 1 Greedy decoding with an explicit KV cache

Input: prompt $x_{1:L}$, maximum new tokens T, end token
$\ell, \mathcal{C} \leftarrow$ FORWARD($x_{1:L}$, positions $0..L{-}1$) ▷ prefill
$y_1 \leftarrow \arg\max \ell$; $n \leftarrow L$
for $t = 2, \ldots, T$ **while** $y_{t-1} \neq$ end token **do**
  $\ell, \mathcal{C} \leftarrow$ FORWARD(y_{t-1}, position n, $\mathcal{C}$)
  $n \leftarrow n + 1$; **assert** $|\mathcal{C}| = n$
  $y_t \leftarrow \arg\max \ell$
end for
return $y_{1:t}$

Algorithm 2 Tiled attention with online softmax

Input: Q, K, V, block sizes B_q, B_k, causal flag
for each query block Q_i **do**
  $m \leftarrow -\infty$; $\ell \leftarrow 0$; $O \leftarrow 0$
  for each key block K_j, V_j **not** entirely in the future **do**
    $S \leftarrow Q_i K_j^\top / \sqrt{d}$; mask diagonal blocks if causal
    update m, ℓ, O with (4)–(6)
  end for
  output block $\leftarrow O/\ell$
end for

and fine-tuned models receive identical prompts. The test split is blocked in the evaluation script unless a flag is passed, and it was evaluated once, after the epoch had been chosen on validation data.

F. KV-Cached Autoregressive Decoding

The decoder calls the model's forward pass with explicit cache management rather than relying on a library switch (Algorithm 1). The prompt is processed once (prefill) with positions $0, \ldots, L-1$. Afterward only the newest token is fed.

Positions. Rotary embeddings [14] rotate keys and queries by their absolute position, and keys are stored in the cache already rotated. The new token must therefore be given the position equal to the number of cached tokens; a wrong position gives wrong outputs although all shapes still match.

Masking. Prefill needs a causal mask. A single new query attends to all cached keys and to itself, so its attention mask is all ones over $n{+}1$ keys; no future token exists yet.

Cache growth. The cache grows by exactly one token per step. The code counts the cached length independently and asserts that the cache length equals the count after every step. For 1,152 tokens the cache occupies $1152 \times 28{,}672$ bytes, about 31.5 MiB.

Numerical differences. The cached path multiplies one row at a time and the uncached path multiplies all rows together, so the accumulation order differs. In half precision this produces small logit differences, which change the greedy token only when the two best logits are almost tied. The uncached reference re-runs the whole sequence at every step with the cache disabled.

G. Tiled Attention with Online Softmax

Scaled dot-product attention for queries Q, keys K, and values V with head dimension d is

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + M\right) V, \tag{3}$$

where, in (3), M is zero for visible positions and $-\infty$ for masked ones. The reference implementation builds the full score matrix, subtracts the row maximum before exponentiation for numerical stability, and normalizes.

The tiled version processes one block of queries at a time and loops over blocks of keys, keeping for every query row a running maximum m, a running normalizer ℓ, and a running weighted sum O. For key block j with scores S_j,

$$m' = \max\big(m,\ \mathrm{rowmax}(S_j)\big), \tag{4}$$

$$P_j = \exp(S_j - m'), \qquad \gamma = \exp(m - m'), \tag{5}$$

$$\ell \leftarrow \gamma\,\ell + \mathrm{rowsum}(P_j), \quad O \leftarrow \gamma\,O + P_j V_j, \quad m \leftarrow m'. \tag{6}$$

After the last block the output is O/ℓ. The factor γ in (5) rescales everything accumulated so far to the new maximum, so the result equals the ordinary softmax up to rounding. Only one tile of scores exists at a time, which reduces the score memory from $O(L_q L_k)$ to $O(B_q B_k)$ for block sizes B_q and B_k (Algorithm 2).

For causal attention the queries are taken to be the last L_q positions of the L_k keys, so that $L_q = 1$ reproduces one cached decoding step. Key blocks entirely in the future of a query block are skipped, and blocks that touch the diagonal are masked inside the tile. A row that is fully masked in a block would give $-\infty - (-\infty)$; a safe maximum of zero is used for such rows. Half-precision inputs are accumulated in single precision, as in fused kernels.

H. Greedy Speculative Decoding

In each round (Algorithm 3) the draft model proposes k tokens greedily using its own cache. The target model then processes the last committed token and the k drafts in a single forward pass and yields its own greedy choice $g_0, \ldots, g_k$ after each input position. The longest prefix of drafts that equals the target's choices is accepted, and the target's next choice is appended: a correction when a draft was rejected, or a free bonus token when all k were accepted. Every committed token is therefore the target's greedy token given the committed prefix, so the output is identical to ordinary greedy decoding; the draft only influences speed.

Cache synchronization is the delicate part. Both caches contain rejected tokens after verification and are cropped. The invariant is that the target cache holds all committed tokens except the last one, and the draft cache holds the committed tokens plus the accepted drafts that it has already processed. The draft model never feeds its last proposal, so its cache is cropped to the old committed length plus $\min(a, k-1)$. Assertions check both lengths in every round. A simple cost model describes the expected speedup. If τ is the mean number

Algorithm 3 One round of greedy speculative decoding

Input: committed tokens c, draft length k

1: draft proposes $d_1, \ldots, d_k$ greedily (own cache)
2: target forward on $[c_{\text{last}}, d_1, \ldots, d_k]$ gives $g_0, \ldots, g_k$
3: $a \leftarrow$ largest $a \leq k$ with $d_j = g_{j-1}$ for all $j \leq a$
4: commit $d_1, \ldots, d_a$ and g_a (correction if $a < k$, bonus if $a = k$)
5: crop target cache to $|c_{\text{new}}| - 1$ tokens
6: crop draft cache to $|c_{\text{old}}| + \min(a, k-1)$ tokens

of tokens committed per round and c the cost of one draft step relative to one target step, then

$$S \approx \frac{\tau}{1 + kc}, \tag{7}$$

ignoring prefill and cache cropping.

I. Application

Fig. 1 shows the workflow of the browser application, which is built with Gradio [15] and the Transformers library [16]. The user pastes a patient note, either with numbered sentences or as free text, and enters criteria under "Inclusion criteria" and "Exclusion criteria" headings, or loads an example. The sentences are numbered from zero, as in the training data. For each criterion the engine builds the prompt, decodes with the selected mode (KV cache, no cache, or speculative), parses the answer, and removes cited identifiers that do not exist in the note.

The explanation of a criterion is assembled only from the predicted label and the cited sentences; no free text is generated. For example, a label *not excluded* without a cited sentence is explained as meaning that nothing in the note triggers the exclusion, not that the note proves its absence.

The overall outcome is an explicit aggregation of criterion-level decisions, not a labelled ground truth, because the dataset only provides criterion-level labels. A criterion *fails* if an inclusion criterion is *not included* or an exclusion criterion is *excluded*; it is *unresolved* if the label is *not enough information* or the answer is invalid; it *passes* otherwise, except that *not applicable* is neutral. If any criterion fails, the outcome is "likely not eligible". Otherwise, if any is unresolved, it is "more information needed", and the unresolved criteria are listed. Otherwise, if at least one passes, it is "likely eligible".

The report is saved as HTML and Markdown and contains the disclaimer, the outcome, a criterion table, the cited evidence text, the explanations, the raw model outputs, and provenance (model, adapter and its source, device, precision, decoding mode, and a fingerprint of the inputs). A benchmark tab runs the same prompts through all three decoding modes after a warm-up pass and reports time, tokens per second, peak memory on a GPU, token equality with the no-cache reference, draft acceptance, and speedups. Saved experiment tables are read from the repository files. Inputs are validated (empty note, missing type headings, excessive length, too many criteria), and model errors are reported as messages without ending the session. The page shows the notice that the system is an educational screening prototype and not a clinical enrollment decision. The application starts with `python app.py`, detects a GPU automatically, runs on the CPU otherwise, and downloads the adapter from the release page if it is not present.

IV. Experimental Setup and Metrics

Hardware and software. All training and measurements ran on a free Kaggle notebook with one NVIDIA Tesla T4 GPU (14.56 GiB usable memory, compute capability 7.5, no bfloat16 and no FlashAttention kernels) and Python 3.13. PyTorch, Transformers [16], and PEFT versions are those of the Kaggle image at the time of the runs. No paid service was used. The application was also run on a personal computer.

Training and evaluation. Training used the configuration of Table IV with seed 42; a repeated run reproduced the validation losses exactly, which shows that training is deterministic. The evaluation used batches of 16 prompts sorted by length.

KV-cache experiments. Correctness compares cached and uncached greedy tokens for four synthetic lengths (input/output of 64/32, 256/64, 512/128, and 1024/128 tokens) and for 30 real validation prompts, in half and single precision, and measures the largest absolute logit difference by teacher forcing. The benchmark covers six input/output length pairs with batch size one, a warm-up run, three repetitions, and synchronized timing. Peak memory is the maximum allocated memory above the loaded weights.

Attention experiments. Eight shape and mask cases (tiny, divisible and non-divisible lengths, queries shorter than keys, non-causal, a single query, large scores, and a model-sized case with 12 heads and head dimension 128) are evaluated in float64, float32, and float16 against a float64 reference. The tolerances are 10^{-9}, 10^{-4}, and 10^{-2}. A hand-sized example checks causality, and a stability demonstration uses scores up to 211. The speed benchmark uses half precision, causal masking, 12 heads, head dimension 128, lengths from 256 to 4096, one warm-up call, and the median of five synchronized runs.

Speculative-decoding experiments. Two workloads are used. Workload A consists of 40 real validation prompts (stop at the end token, at most 64 new tokens). Workload B consists of two synthetic prompts of 256 and 768 tokens with exactly 96 new tokens. Draft lengths $k \in \{2, 3, 4, 6\}$ and three drafts are compared: the target itself (an oracle control), the base 0.5B model, and the LoRA-fine-tuned 0.5B model. The draft was fine-tuned with the recipe of Table IV (8,798,208 trainable parameters, validation loss 0.1495, 0.1333, and 0.1396 for the three epochs). Every configuration is compared token by token with ordinary cached greedy decoding, with two repetitions in half precision and one in single precision. The assisted-generation function of the Transformers library is run as an independent reference at $k = 4$.

Reproducibility. Code, split manifest, logs, and scripts are in the repository https://github.com/Maryam-Faatima/clinical_trial_matching_llm. The adapter is available at https://github.com/Maryam-Faatima/clinical_trial_matching_llm/releases/download/v1.0/adapter_epoch_3.zip. A recorded demonstration is at VIDEO-LINK-HERE.

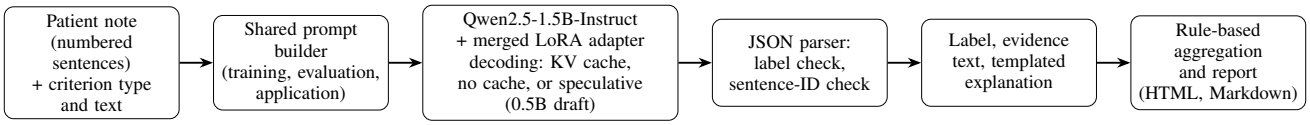


Fig. 1. Architecture of the screening system. The same prompt builder is used for training, evaluation, and the browser application; invalid model outputs are marked unresolved and are never repaired.

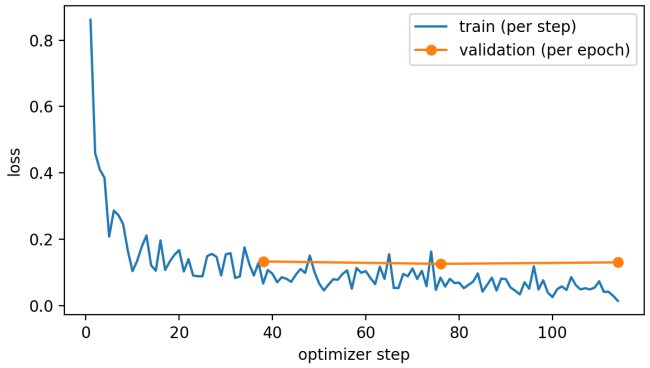


Fig. 2. Training loss per optimizer step and validation loss per epoch for the LoRA fine-tuning of Qwen2.5-1.5B-Instruct.

TABLE V
VALIDATION RESULTS BEFORE AND AFTER FINE-TUNING ($n = 206$)

Model	Val. loss	Acc.	Macro-F1	Evid. micro-F1
Base (no adaptation)	–	0.107	0.118	0.128
Epoch 1	0.1327	0.728	0.353	0.459
Epoch 2	0.1255	0.752	0.444	0.541
Epoch 3 (selected)	0.1300	0.767	0.574	0.558

V. RESULTS AND DISCUSSION

A. Fine-Tuning and Epoch Selection

Fig. 2 shows the training and validation losses. The loss accumulated over 16 examples falls from about 0.10–0.17 in the first epoch to 0.03–0.08 in the third, while the validation loss stays between 0.1255 and 0.1327 and is lowest after epoch 2. Training took 834 s with a peak memory of 7.95 GiB. Table V shows that the validation macro-F1 nevertheless increases at every epoch (0.353, 0.444, 0.574), whereas the lowest loss occurs at epoch 2. Loss and macro-F1 measure different things; loss is dominated by the frequent labels, and macro-F1 rewards the rare ones. Epoch 3 was selected because macro-F1 is the metric of interest. The validation set contains only two *excluded* and three *not included* examples, so this selection is noisy, which is noted in Section VI.

B. Held-Out Test Results

Table VI gives the test results and Fig. 3 visualizes the change. Fine-tuning raises overall accuracy from 0.117 to 0.743 and macro-F1 from 0.142 to 0.536. A classifier that always answers *not excluded* would reach an accuracy of 0.442 (91 of 206), so the fine-tuned model is well above this baseline. Compared with validation (0.767 accuracy, 0.574 macro-F1), the test scores are slightly lower, which gives no sign that the epoch selection overfitted the validation patients. Macro-F1 is computed over the five labels that occur in the test gold labels or predictions; the test set has no *excluded* row and the fine-tuned model never predicts that label. The base model predicts *excluded* 84 times, always wrongly, so its macro-F1 is 0.142 over six labels and 0.171 when restricted to the gold labels.

TABLE VI
TEST RESULTS ($n = 206$; INCLUSION $n = 75$, EXCLUSION $n = 131$)

Model	Subset	Acc.	Macro-F1	Evidence micro		
				P	R	F1
Base	Overall	0.117	0.142	0.075	0.362	0.124
Fine-tuned	Overall	0.743	0.536	0.530	0.564	0.546
Base	Inclusion	0.227	0.195	0.323	0.339	0.331
Fine-tuned	Inclusion	0.800	0.433	0.588	0.758	0.662
Base	Exclusion	0.053	0.057	0.033	0.406	0.062
Fine-tuned	Exclusion	0.710	0.400	0.300	0.188	0.231

TABLE VII
PER-CLASS RESULTS OF THE FINE-TUNED MODEL ON THE TEST SPLIT

Label	Precision	Recall	F1	Support
included	1.000	0.815	0.898	27
not included	0.000	0.000	0.000	7
not enough information	0.717	0.576	0.639	66
not applicable	0.750	0.200	0.316	15
not excluded	0.709	0.989	0.826	91
excluded	–	–	–	0

Inclusion criteria are easier (accuracy 0.800) than exclusion criteria (0.710), but the macro-F1 values are low for both (0.433 and 0.400) because several labels are never predicted correctly. Table VII and Table VIII show where the errors are. The labels *included* (F1 0.898) and *not excluded* (F1 0.826) are handled well. The label *not enough information* reaches an F1 of 0.639 overall, with a recall of 1.000 for inclusion criteria but 0.000 for exclusion criteria. *Not included* (0 of 7) and *not applicable* (3 of 15) are largely missed.

Fig. 4 contrasts the confusion matrices of the base and fine-tuned models. The base model is format-compliant (no invalid outputs) but misreads the label semantics: of the 91 *not excluded* rows it answers *excluded* 57 times and *not applicable* 33 times, and it never predicts *not enough information*. This behavior is the reason for supervised adaptation. The fine-tuned model confuses the same rare classes in a different way, by collapsing them into the majority decisions.

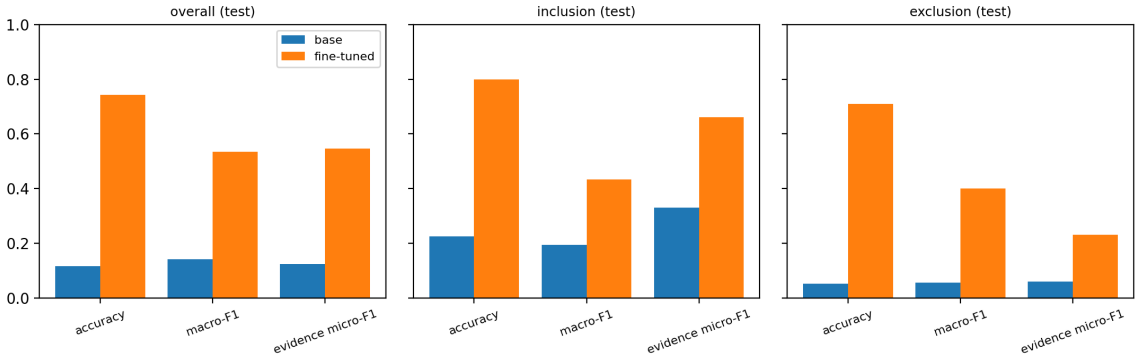


Fig. 3. Accuracy, macro-F1, and evidence micro-F1 of the base and fine-tuned models on the test split, overall and for inclusion and exclusion criteria.

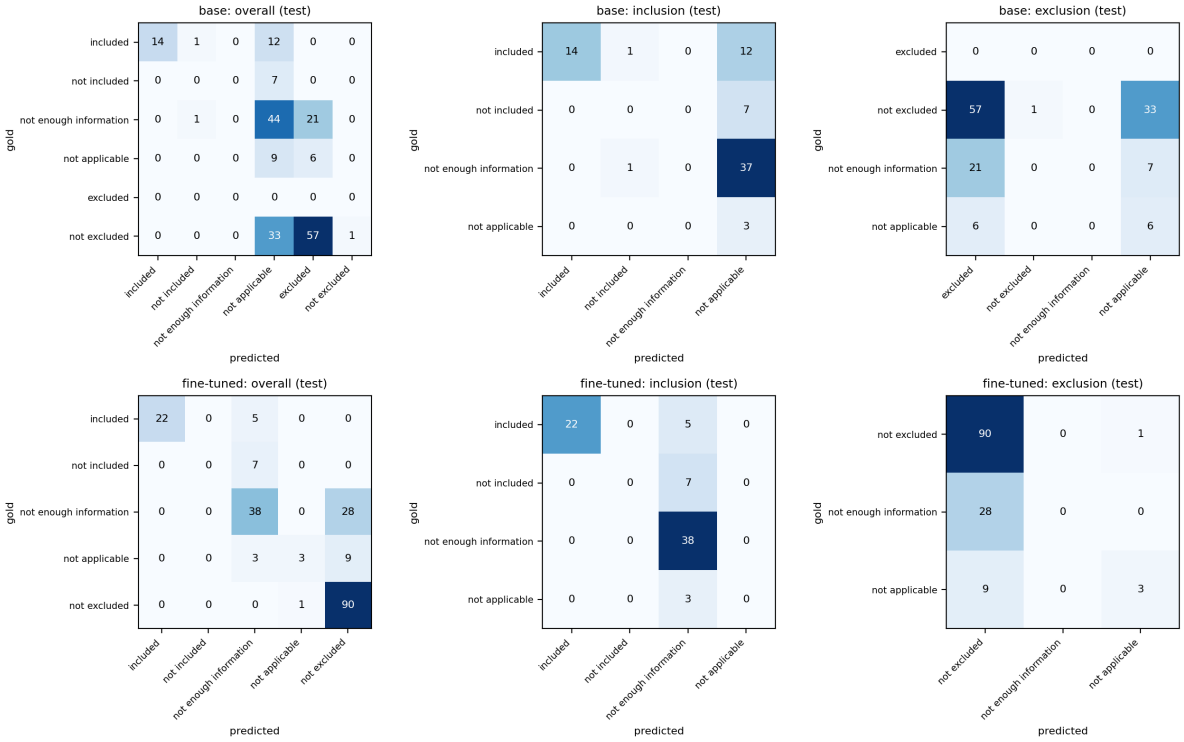


Fig. 4. Confusion matrices on the test split for the base model (top row) and the fine-tuned model (bottom row): overall, inclusion criteria, and exclusion criteria. Rows are gold labels and columns are predictions.

TABLE VIII
CONFUSION MATRIX OF THE FINE-TUNED MODEL ON THE TEST SPLIT (ROWS: GOLD, COLUMNS: PREDICTED). ABBREVIATIONS: INC. = INCLUDED, N.INC. = NOT INCLUDED, NEI = NOT ENOUGH INFORMATION, N/A = NOT APPLICABLE, N.EXC. = NOT EXCLUDED.

Gold \ Pred.	inc.	n.inc.	NEI	N/A	n.exc.
included	22	0	5	0	0
not included	0	0	7	0	0
NEI	0	0	38	0	28
N/A	0	0	3	3	9
not excluded	0	0	0	1	90

C. Evidence Localization

The pooled evidence precision, recall, and F1 of the fine-tuned model are 0.530, 0.564, and 0.546 (Table VI), against 0.075, 0.362, and 0.124 for the base model. The macro F1 over rows with gold evidence is 0.581 (base 0.157), and over all rows it is 0.798 (base 0.376). On rows without gold evidence the fine-tuned model correctly cites nothing in 89.0% of cases, whereas the base model does so in 46.9%. Evidence for inclusion criteria (micro-F1 0.662) is much better than for exclusion criteria (0.231). The comparison of identifier sets is strict: a cited sentence that overlaps in content with a gold sentence but has a different identifier counts as an error,

TABLE IX
ERRORS OF THE FINE-TUNED MODEL ON THE TEST SPLIT BY TYPE

Error type	Count
Missing information treated as proof	28
False *not enough information*	15
Not-applicable confusion	10
Wrong decision polarity	0
Label errors in total	53
Correct label, different evidence set	37

so these values are lower bounds on the usefulness of the citations.

D. Error Analysis

There are 53 label errors on the test split. Table IX groups them, and in addition 37 rows have the correct label but a different evidence set. Table X lists twelve representative cases, three per category, drawn with a fixed random seed from the errors of the fine-tuned model.

Missing information treated as proof (28 cases) is the dominant failure. For exclusion criteria with gold label *not enough information*, the model returned *not excluded* in all 28 cases without citing a sentence (cases 4 to 6): the note gives no information on MRI metal screening, on recent immunosuppressive treatment, or on cyclophosphamide, and the model read this silence as a negative answer. This is exactly the error that the task warns against, and it is a consequence of the class imbalance (483 of 658 exclusion rows are *not excluded*) and of the small training set.

False insufficient information (15 cases) is the opposite, conservative error. In case 1 the expert accepted a consent-and-compliance sentence as evidence of the ability to take oral medication, whereas the model abstained. In cases 2 and 3 the experts judged the procedure criteria as not met without citing any sentence, and the model abstained instead of answering *not included*.

Not-applicable confusion (10 cases) arises when a criterion does not concern the patient, such as pregnancy (case 7) or a tonsillectomy (case 9) in the same patient; the model answered *not excluded* instead of recognizing that the criterion is not applicable.

Right label, different evidence (37 cases) shows spurious or excessive citation. In case 10 the model cited an allergic-rhinitis sentence for a skin-test criterion that experts judged to have no evidence. In case 11 it cited the gold sentence plus one additional sentence. In case 12 it cited three symptom sentences for an exclusion criterion that experts judged as not excluded without evidence. No error of the type *wrong decision polarity* occurred on the test split. On the validation split the same categories appear (45, 26, 14, 6, and 2 cases for evidence, missing information, false abstention, not-applicable confusion, and polarity), so the pattern is stable.

E. KV-Cached Decoding

Correctness. Cached and uncached greedy decoding produced identical tokens in all four synthetic cases and in all 30 real validation prompts, in half and in single precision, so no mismatch had to be investigated. The largest absolute logit difference between the two paths is between 0.040 and 0.056 in half precision and between 1×10^{-4} and 2×10^{-4} in single precision. The reduction by a factor of about 300 shows that the differences are rounding noise and not a logic error in the cache.

Speed and memory. Table XI and Fig. 5 show that cached decoding runs at about 22–26 tokens per second at every length, at about 39 ms per generated token, whereas the uncached speed falls from 22.9 to 3.1 tokens per second as the sequence grows. The speedup therefore rises from 1.1 to 8.0 times. The uncached cost per step grows with the sequence length, and the cached cost does not at these lengths. The constant 39 ms per token indicates that single-sequence decoding on this GPU is dominated by kernel-launch and software overhead, not by arithmetic. Peak memory is similar for both paths (10–188 MiB above the weights) and is dominated by the activations of the largest forward pass; the cache itself is only about 31.5 MiB at 1,152 tokens. The cache saves time, not memory. Prefill time grows with the prompt from 46 to 275 ms.

F. Tiled Attention

All 24 correctness tests (eight cases in three precisions) passed (Table XII). The hand-sized example gives identical output rows for the reference and the tiled implementation, the first row equals the first value vector, and changing the last token does not change any earlier output, which confirms causality. In the stability demonstration with scores up to 211, a softmax without maximum subtraction produced non-finite values, whereas the tiled implementation stayed finite with a maximum error of 7.3×10^{-6} against float64. In half precision the error of about 10^{-3} is mostly the rounding of the final output to 16 bits, and the reference and tiled versions are equally accurate.

Table XIII and Fig. 6 report speed and memory. The memory of the reference grows by about four times when the length doubles (2352 MiB at 4096 tokens), reflecting the quadratic score matrix. The tiled version needs only 18.8 MiB at 4096 tokens, 125 times less, with only a slow increase. It is, however, about 2.9 times slower than the reference and about 55 times slower than the fused memory-efficient backend of PyTorch (191 against 3.5 ms). The educational implementation is a Python loop with hundreds of block iterations, each launching about ten small GPU operations whose intermediate tiles pass through global memory. A fused kernel runs the whole loop in one launch and keeps the tiles in on-chip memory. The FlashAttention backend was unavailable because it requires compute capability 8.0 or higher, whereas the T4 has 7.5. The tiling therefore saves memory, but only fusion saves time.

G. Speculative Decoding

Correctness. Every configuration produced tokens identical to ordinary cached greedy decoding: 40 of 40 real prompts

TABLE X
TWELVE TEST ERRORS OF THE FINE-TUNED MODEL (ANNOTATION IDENTIFIER, CRITERION TYPE, GOLD AND PREDICTED LABEL)

No.	ID	Type	Criterion (shortened)	Gold	Predicted
1	385	incl.	Ability to take and retain oral medication	included	not enough info.
2	162	incl.	Open abdominal aortic aneurysm repair	not included	not enough info.
3	161	incl.	Elective carotid endarterectomy	not included	not enough info.
4	418	excl.	Failure to pass MRI metal screening	not enough info.	not excluded
5	393	excl.	Anti-rheumatic or immunosuppressive drugs within 3 months	not enough info.	not excluded
6	392	excl.	Intravenous cyclophosphamide within 6 months	not enough info.	not excluded
7	697	excl.	Pregnant	not applicable	not excluded
8	693	excl.	Suspected sleep disorder other than sleep apnea	not applicable	not excluded
9	683	excl.	Previous tonsillectomy	not applicable	not excluded
10	840	incl.	Negative allergy skin test	not enough info. (evidence differs)	same label, cites sentence 1
11	307	incl.	Liver-damage patient selection (long text)	not enough info. (evidence differs)	same label, cites sentences 3 and 4
12	769	excl.	Infection of the deep upper-airway tissues	not excluded (evidence differs)	same label, cites sentences 0 to 2

TABLE XI
KV-CACHED VERSUS UNCACHED GREEDY DECODING (TESLA T4, HALF PRECISION, BATCH SIZE 1; TOKENS IDENTICAL IN ALL ROWS)

Input + output tokens	Uncached tok/s	Cached tok/s	Speedup	Prefill (ms)	Decode (ms/token)	Peak MiB (unc./cached)
128 + 32	22.9	25.7	1.1×	46	38.7	10 / 12
128 + 128	20.1	25.8	1.3×	47	38.8	18 / 13
512 + 32	6.9	24.3	3.5×	113	38.9	54 / 62
512 + 128	6.6	25.3	3.8×	115	39.0	69 / 62
1024 + 32	3.3	21.6	6.6×	275	38.9	161 / 180
1024 + 128	3.1	24.7	8.0×	274	38.6	188 / 180

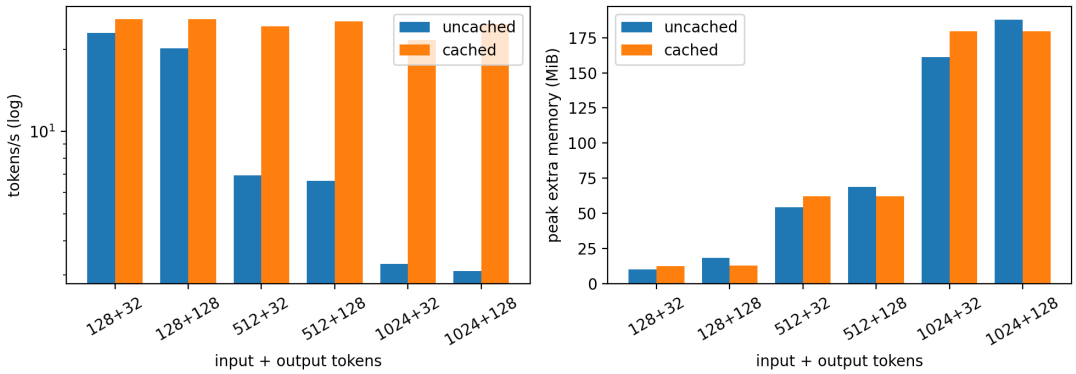


Fig. 5. Throughput (left, logarithmic axis) and peak extra memory (right) of uncached and KV-cached greedy decoding for the six length configurations.

TABLE XII
ABSOLUTE ERROR OF TILED ATTENTION AGAINST A FLOAT64 REFERENCE (RANGE OVER EIGHT CASES)

Precision	Tolerance	Max error	Mean error
float64	10^{-9}	1.1×10^{-16}–4.4×10^{-15}	2.3×10^{-17}–2.3×10^{-16}
float32	10^{-4}	1.4×10^{-8}–1.3×10^{-5}	5.8×10^{-9}–3.7×10^{-7}
float16	10^{-2}	6.0×10^{-5}–9.7×10^{-4}	1.3×10^{-5}–1.1×10^{-4}

for each draft and each k and 2 of 2 synthetic prompts in half precision, and 20 of 20 and 2 of 2 in single precision. The independent assisted-generation reference also matched in 40 of 40 prompts. When the target is used as its own draft, the acceptance is exactly 1.00 for every k, which tests the verification, rollback, and cache logic.

Efficiency. Table XIV and Fig. 7 show that no configuration is faster than ordinary decoding, which runs at 24.9 tokens per second (610 ms per prompt, about 15 generated tokens). The best case is the fine-tuned draft with $k = 2$ at 1.00 times, with an acceptance of 0.96. The base draft is accepted less often (0.73–0.84) and yields 0.76–0.91 times. The fine-tuned draft is accepted more often on the real task (0.93–0.96) because

TABLE XIII
ATTENTION SPEED AND MEMORY (TESLA T4, HALF PRECISION, CAUSAL, 12 HEADS, HEAD DIMENSION 128; TIME IN MS, EXTRA MEMORY IN MIB)

Length	Reference ms	Reference MiB	Tiled ms	Tiled MiB	SDPA math ms	SDPA mem-eff. ms	SDPA mem-eff. MiB
256	0.59	12.0	1.61	7.5	0.62	0.09	0.8
512	1.28	42.0	4.51	8.3	1.07	0.14	1.5
1024	4.44	156.0	14.70	9.8	3.66	0.33	3.0
2048	16.87	600.1	51.13	12.8	13.91	1.12	6.0
4096	66.97	2352.2	191.28	18.8	55.25	3.48	12.0

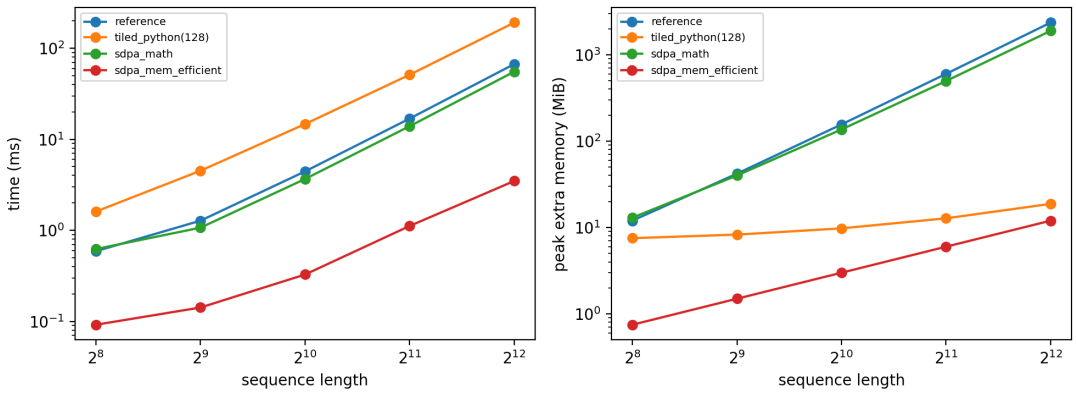


Fig. 6. Time (left) and peak extra memory (right) of the reference, tiled, and PyTorch attention backends against sequence length, on logarithmic axes.

it has learned the output format and label vocabulary. On the synthetic long prompts, which are free text, the acceptance drops to 0.33–0.66 and the speed to 0.49–0.87 times, and the fine-tuned draft is no better than the base draft (acceptance 0.49 against 0.57 at $k = 3$), because it was trained only on short JSON answers. The library reference reaches 0.90 times (base draft) and 0.98 times (fine-tuned draft) at $k = 4$, against 0.83 and 0.96 times for the implementation here, which is a few percent slower. Single precision gives 0.57–0.83 times, so the result is not a rounding artefact. Loading the draft adds 0.94 GiB to the peak memory in half precision (2,990 to 3,934 MiB) and 1.9 GiB in single precision (5,954 to 7,839 MiB).

Why there is no speedup. Equation (7) explains the result. Solving it for c with the measured speedups and tokens per round gives $c \approx 0.80$–0.84 for all eight real-prompt configurations with the base and the fine-tuned draft, and $c \approx 1.03$–1.05 for the oracle, where the draft equals the target and c should be close to one; the oracle supports the model. A draft step costs about 0.8 of a target step because decoding at batch size one is overhead-bound (Section V-E) and the 0.5B model still has 24 layers. With $c \approx 0.8$, even a perfect acceptance cannot pay for the drafting. If a draft were much cheaper, for example $c = 0.3$, the same model with an acceptance of 0.93 and $k = 4$ would give about $4.14/(1 + 1.2) \approx 1.9$ times; this is a calculation, not a measurement. The short outputs of this task (about 15 tokens) also leave few rounds in which the method can pay off, and the prefill cost is not reduced.

TABLE XIV
SPECULATIVE DECODING ON 40 REAL VALIDATION PROMPTS (BASELINE: CACHED GREEDY, 24.9 TOK/S; ACC. = DRAFT ACCEPTANCE; τ = TOKENS PER ROUND)

Draft	k	Acc.	τ	tok/s	Speedup
Target (oracle)	2	1.00	2.70	21.8	0.88
	3	1.00	3.40	20.6	0.83
	4	1.00	4.46	21.5	0.86
	6	1.00	6.37	21.6	0.87
Base 0.5B	2	0.84	2.38	22.7	0.91
	3	0.84	2.94	21.2	0.85
	4	0.80	3.54	20.6	0.83
	6	0.73	4.50	19.0	0.76
Fine-tuned 0.5B	2	0.96	2.61	24.8	1.00
	3	0.95	3.28	23.5	0.95
	4	0.93	4.14	23.9	0.96
	6	0.93	5.79	23.8	0.96

H. Application

Fig. 8 shows the application. A screening of an example note with seven criteria takes about 0.6 s per criterion on the T4 with the KV-cache mode, and every criterion is shown with its label, status, cited sentences, and explanation. The application was tested with automated tests of the parsing, decision, aggregation, and report logic (five test groups), and by hand with a normal example, a case with missing information, the three decoding modes (the decisions must be identical, and only the times differ), the benchmark tab, and invalid inputs (empty note, missing type headings, excessive length), after each of which the application recovered. Unseen

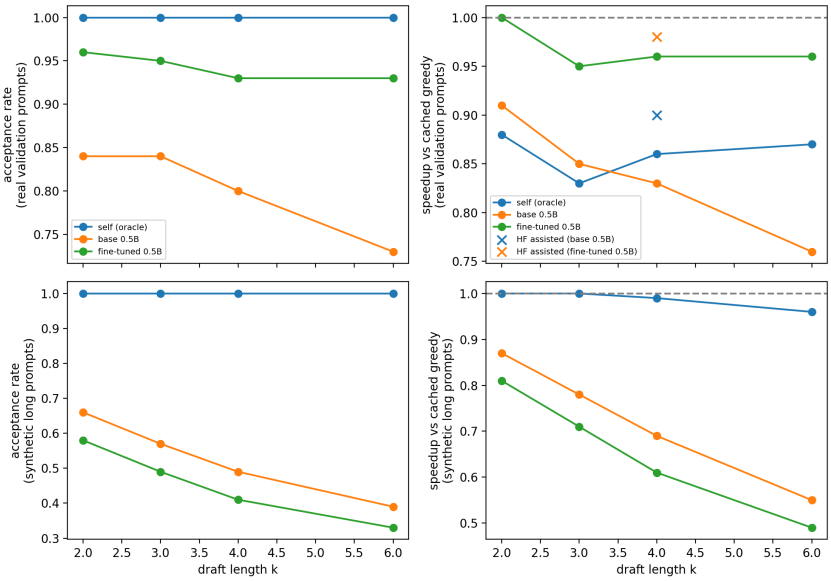


Fig. 7. Draft acceptance rate (left) and speedup over cached greedy decoding (right) against draft length k, for real validation prompts (top) and synthetic long prompts (bottom). Crosses mark the library assisted-generation reference at $k = 4$.

notes and criteria entered by the user are processed in the same way.

VI. Limitations

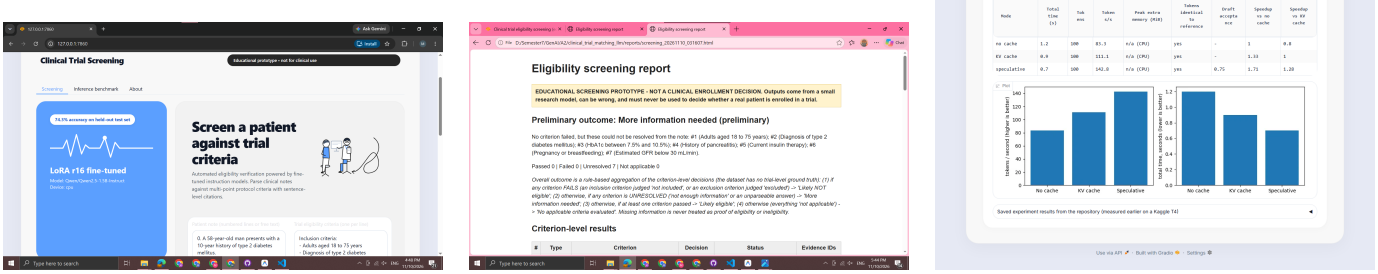
- **Data and statistics.** The training set has 602 rows from 37 patients, and the test set has 8 patients with 0 *excluded* and 7 *not included* examples. Each result comes from one run and one seed, without confidence intervals, so differences in rare classes and in macro-F1 are noisy. The patient notes are synthetic vignettes, not real records.
- **Model quality.** The model treats missing information as proof for exclusion criteria and misses rare labels. Class-weighted loss, oversampling, a larger model, more epochs, and a hyperparameter search were not tried because of the time limit.
- **Evidence metric.** Exact identifier matching is strict and understates useful citations.
- **Epoch selection.** Validation macro-F1 rests on two or three examples for rare classes.
- **Hardware.** All measurements come from one T4 GPU at batch size one, where decoding is overhead-bound. The tiling and speculative-decoding conclusions may differ on other hardware, with larger models, or with batching. FlashAttention kernels could not be tested.
- **Lost artifact.** The adapter and training log of the fine-tuned draft model were lost when a notebook session ended. Its speculative-decoding results are taken from the saved console log; the exact command to retrain the draft is in the repository, and training was shown to be deterministic.
- **Precision.** Reported accuracies were measured in half precision on a GPU. On a CPU in single precision, rare differences in outputs are possible.
- **Overall verdict.** The trial-level outcome is a rule-based aggregation, not validated against any ground truth, and the application has not been validated for clinical use.

VII. Conclusion

A 1.5-billion-parameter open-weight model, fine-tuned with LoRA on 602 expert-labelled examples using free hardware, raised criterion-level accuracy from 0.117 to 0.743 and macro-F1 from 0.142 to 0.536 on held-out patients, and it cites supporting sentences with an F1 of 0.546. Its main weakness is to read missing information as a negative answer for exclusion criteria, which is a risk for any use beyond education. Three inference mechanisms were implemented and verified against references. The KV cache gives identical output and up to 8.0 times higher throughput; tiled online-softmax attention is exact to rounding and 125 times smaller in memory at 4096 tokens but slower without a fused kernel; and speculative decoding is exact but gives no speedup for this model pair on a T4, because the draft step costs about 80% of a target step. The browser application makes these results inspectable through per-criterion evidence, a rule-based outcome, an exportable report, and a live benchmark. Future work includes class-balanced training, a cheaper draft model, evaluation on more patients, and tests on hardware with fused attention kernels.

Appendix A
AI Assistance Statement

An AI assistant (Claude, Anthropic) was used to plan the work, draft and debug source code, and draft the text and the LaTeX source of this paper. The author executed all training runs and experiments, checked every reported number against the saved logs and result files, tested the code, and takes responsibility for the content. Existing libraries were used for



(a) (b) (c)

Fig. 8. Browser application. (a) Screening tab with criterion-level results and cited evidence. (b) Exported HTML report with disclaimer, outcome, and provenance. (c) Benchmark tab comparing the three decoding modes live.

tokenization, model loading, adapter training, metrics, plotting, and the user interface (PyTorch, Transformers, PEFT, scikit-learn, NumPy, pandas, Matplotlib, and Gradio). The assisted-generation function of the Transformers library served only as a reference for testing speculative decoding. The TrialGPT code repository was not run; only the public dataset was used.

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, 2024, doi: 10.1038/s41467-024-53081-z.

[2] M. Nievas, A. Basu, Y. Wang, and H. Singh, "Distilling large language models for matching patients to clinical trials," *Journal of the American Medical Informatics Association*, vol. 31, no. 9, 2024, doi: 10.1093/jamia/ocae073.

[3] National Center for Biotechnology Information, "TrialGPT criterion annotations dataset," Hugging Face Datasets, ncbi/TrialGPT-Criterion-Annotations, 2024, https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[6] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," arXiv preprint arXiv:1805.02867, 2018.

[7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[8] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[10] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023.

[11] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," arXiv preprint arXiv:2302.01318, 2023.

[12] Qwen Team, "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.

[13] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[14] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," arXiv preprint arXiv:2104.09864, 2021.

[15] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, "Gradio: Hassle-free sharing and testing of ML models in the wild," arXiv preprint arXiv:1906.02569, 2019.

[16] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.